

./02-architecture/DOC-09-RBAC.md

DOC-09: RBAC & Permission Architecture

Version: 1.0 | **Status:** APPROVED | **Date:** 2026-08-27 **Purpose:** Detail the Role-Based Access Control system.

Owner: Security Architect

Architecture

User -> Membership -> Role -> Permissions

- **Permissions Structure:** resource.action (e.g., users.view, inventory.adjust).
- **Standard Actions:** view, create, update, delete, approve, reject, export, import, print.

Role Types

1. **System Roles:** Owner, Admin, Viewer. These are immutable and protected.
2. **Custom Roles:** User-defined roles (e.g., “Purchase Manager”) configured via UI.

Permission Enforcement

- **Backend Authority:** The backend is the absolute authority for authorization. API endpoints use NestJS Guards to verify permissions.
- **Frontend Visibility:** The frontend UI conditionally renders buttons, menus, and pages based on the user’s permissions, but this is for UX only, not security.
- **Caching:** Permissions are cached in Redis (user:{id}:permissions) to prevent repeated DB hits. Cache is invalidated on role/permission updates.